Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 1:

Classical Cryptography: Caesar Cipher and Caesar Cipher with Permutation

Elaborated:
Ilie Covali — gr. FAF-243

Verified: asist. univ. Zaica Maia

Chișinău – 2026

Contents

1 Short Theory **2**
1.1 Introduction to Classical Cryptography . . . 2
1.2 Alphabet Definition . . . 2
1.3 The Standard Caesar Cipher . . . 2
1.3.1 Encryption and Decryption Formulas . . . 2
1.3.2 Key Space Analysis . . . 2
1.4 The Caesar Cipher with Permutation . . . 3
1.4.1 Building the Permuted Alphabet . . . 3
1.4.2 Encryption and Decryption Formulas . . . 3
1.4.3 Proof of Invertibility . . . 3
1.4.4 Key Space Analysis . . . 4

2 Implementation **5**
2.1 Alphabet Encoding and Diacritic Handling . . . 5
2.2 Input Validation Module . . . 5
2.3 Building the Permuted Alphabet . . . 6
2.4 Encryption and Decryption Routines . . . 6
2.4.1 Task 1: Standard Caesar Cipher . . . 6
2.4.2 Task 2: Caesar Cipher with Permutation . . . 7

3 Results **9**
3.1 Task 1: Standard Caesar Cipher Execution . . . 9
3.1.1 Encryption Demonstration . . . 9
3.1.2 Decryption Demonstration . . . 9
3.1.3 Input Validation: Out-of-Bounds Key . . . 9
3.1.4 Input Validation: Invalid Characters . . . 10
3.2 Task 2: Caesar Cipher with Permutation Execution . . . 11
3.2.1 Encryption Demonstration . . . 11
3.2.2 Decryption Demonstration . . . 11
3.2.3 Input Validation: Keyword Too Short . . . 11
3.2.4 Input Validation: Invalid Key . . . 12

4 Conclusions **13**

5 Source Code **13**
5.1 Repository File Structure . . . 13

1 Short Theory

1.1 Introduction to Classical Cryptography

Classical cryptography primarily relies on two fundamental classes of operations: *transposition* (rearranging the order of plaintext characters) and *substitution* (replacing plaintext characters with alternative symbols, letters, or numbers according to a predetermined rule).

Monoalphabetic substitution ciphers represent one of the oldest forms of encryption, in which each letter of the plaintext is deterministically replaced by a corresponding letter from a fixed ciphertext alphabet throughout the entire message. In this laboratory work, we investigate two variations of monoalphabetic substitution over the Romanian alphabet: the standard Caesar cipher and the Caesar cipher with a keyword permutation.

1.2 Alphabet Definition

The implementations operate on the Romanian alphabet comprising $N = 31$ characters, ordered as follows:

$$\mathscr{A} = [\text{a, ă, â, b, c, d, e, f, g, h, i, î, j, k, l, m, n, o, p, q, r, s, ș, t, ț, u, v, w, x, y, z}] \tag{1}$$

Let $I : \mathscr{A} \to \mathbb{Z}_N = \{0, 1, \ldots, N-1\}$ be the bijection mapping each character to its zero-based position in $\mathscr{A}$, and let $\mathscr{A}[i]$ denote the character corresponding to index $i \in \mathbb{Z}_N$.

1.3 The Standard Caesar Cipher

The Caesar cipher, historically attributed to Julius Caesar, shifts each character of the plaintext forward in the alphabet by a fixed integer key k.

1.3.1 Encryption and Decryption Formulas

For a plaintext letter $P_i \in \mathscr{A}$ and a numeric key $k \in \mathbb{Z}_N$, encryption is defined by modular addition:

$$C_i = E(P_i, k) = \mathscr{A}\left[(I(P_i) + k) \pmod N\right] \tag{2}$$

Decryption performs the inverse operation using modular subtraction:

$$P_i = D(C_i, k) = \mathscr{A}\left[(I(C_i) - k) \pmod N\right] \tag{3}$$

1.3.2 Key Space Analysis

Because the transformation is entirely governed by the shift parameter $k \pmod N$, any shift of $k \equiv 0 \pmod N$ yields the identity mapping ($C_i = P_i$). Consequently, the set of non-trivial encryption keys is:

$$\mathscr{K}_{\text{Caesar}} = \{1, 2, \ldots, N-1\} \tag{4}$$

For the Romanian alphabet with $N = 31$, the key space consists of exactly:

$$|\mathscr{K}_{\text{Caesar}}| = 31 - 1 = 30 \text{ possible keys} \tag{5}$$

Due to this minute key space, the Caesar cipher offers negligible computational security. An adversary can perform an exhaustive brute-force search over all 30 keys in microseconds without requiring computational assistance.

1.4 The Caesar Cipher with Permutation

The Caesar cipher with a permutation (keyword Caesar cipher) combines a keyword-based permutation of the alphabet with a secondary numeric shift k. This addresses a significant structural defect of the standard Caesar cipher: the predictable sequential arrangement of ciphertext letters.

1.4.1 Building the Permuted Alphabet

Two secret keys are provided:

1. A string keyword K satisfying a minimum length requirement ($\text{length}(K) \geq 7$) and containing characters exclusively from $\mathscr{A}$.
2. A numeric key $k \in \{1, 2, \ldots, N-1\}$.

The permuted alphabet Π of size N is derived as follows:

1. **Deduplication**: Duplicate occurrences of letters in K are eliminated while retaining the initial occurrence order, producing sequence $K' = \text{dedup}(K)$.
2. **Alphabet Completion**: The remaining characters from the standard alphabet $\mathscr{A}$ that do not appear in K' are appended in their natural order:

$$\Pi = K' \,||\, (\mathscr{A} \setminus K') \tag{6}$$

1.4.2 Encryption and Decryption Formulas

Let $I_{\mathscr{A}}(x)$ denote the index of character x in the standard alphabet $\mathscr{A}$, let $\Pi[j]$ denote the j-th character of the permuted alphabet Π, and let $I_{\Pi}(y)$ denote the index of character y within Π.

For a plaintext character $P_i \in \mathscr{A}$:

$$C_i = E(P_i, k, \Pi) = \Pi\big[(I_{\mathscr{A}}(P_i) + k) \pmod N\big] \tag{7}$$

Decryption proceeds by locating the ciphertext character in the permuted alphabet, shifting back by k, and retrieving the plaintext character from the standard alphabet:

$$P_i = D(C_i, k, \Pi) = \mathscr{A}\big[(I_{\Pi}(C_i) - k) \pmod N\big] \tag{8}$$

1.4.3 Proof of Invertibility

Let $m = I_{\mathscr{A}}(P_i)$. The encryption step yields:

$$C_i = \Pi[c], \quad \text{where } c = (m + k) \pmod N \tag{9}$$

Since Π is a bijective permutation of $\mathscr{A}$, the index of C_i in Π is:

$$I_{\Pi}(C_i) = c = (m + k) \pmod N \tag{10}$$

Applying the decryption formula:

$$\begin{aligned} (I_{\Pi}(C_i) - k) \pmod N &= \big(((m + k) \pmod N) - k\big) \pmod N \\ &= (m + k - k) \pmod N = m \pmod N \end{aligned} \tag{11}$$

Since $\mathscr{A}[m] = P_i$, we obtain $D(E(P_i, k, \Pi), k, \Pi) = P_i$, proving that the cipher is lossless and fully invertible.

1.4.4 Key Space Analysis

The theoretical upper bound on any monoalphabetic substitution cipher over $N = 31$ symbols corresponds to the order of the symmetric group S_{31}:

$$|S_{31}| = 31! \approx 8.2228 \times 10^{33} \tag{12}$$

In the Caesar cipher with permutation, the key space is constrained by the keyword generation process combined with the numeric shift $k \in \{1, \ldots, N-1\}$. For a keyword consisting of m unique characters ($7 \leq m \leq 31$), the number of permutations of length m drawn from 31 letters is given by the partial permutations $P(31,m) = \frac{31!}{(31-m)!}$.

Taking only the minimum required length $m = 7$:

$$P(31,7) = \frac{31!}{24!} = 31 \times 30 \times 29 \times 28 \times 27 \times 26 \times 25 = 13,289,628,000 \approx 1.33 \times 10^{10} \tag{13}$$

Coupled with the 30 non-trivial numeric shifts, the minimum key space is:

$$|\mathscr{K}_{\text{perm}}| \geq P(31,7) \times (N-1) = 1.33 \times 10^{10} \times 30 \approx 3.986 \times 10^{11} \text{ combinations} \tag{14}$$

When longer keywords are used, the key space exceeds 10^{15} combinations. This renders naive exhaustive search computationally intractable by hand and substantially raises the bar against brute-force attacks compared to standard Caesar. However, because the cipher remains a monoalphabetic substitution, letter frequencies are preserved, meaning the cipher remains vulnerable to statistical cryptanalysis.

2 Implementation

The laboratory tasks were implemented in Python 3 across two dedicated modules: `caesar1.py` (standard Caesar cipher) and `caesar2.py` (Caesar cipher with permuted alphabet). Both modules adhere to defensive programming principles, ensuring input sanitization and descriptive error feedback.

2.1 Alphabet Encoding and Diacritic Handling

The Romanian alphabet contains five characters with diacritics (ă, â, î, ș, ț). To prevent character corruption and ensure faithful indexing, the alphabet is defined as an explicit UTF-8 list of distinct string characters:

```
alphabet = [
    "a", "ă", "â", "b", "c", "d", "e", "f", "g", "h", "i", "î", "
       j", "k",
    "l", "m", "n", "o", "p", "q", "r", "s", "ș", "t", "ț", "u", "
       v", "w",
    "x", "y", "z"
]
```

Comment: The 31 characters are assigned sequential indices from 0 to 30. Input text is converted to lowercase upon arrival, preserving whitespace characters to maintain sentence readability.

2.2 Input Validation Module

A robust validation layer ensures that only authorized inputs reach the cryptographic core:

```
def validate_key(key_input):
    try:
        key = int(key_input)
        if 1 <= key < len(alphabet):
            return key
        else:
            print(f"Error: Invalid key! The numeric key must be
               an integer between 1 and {len(alphabet) - 1}.")
            return None
    except ValueError:
        print("Error: Invalid key! Key must be an integer number.
           ")
        return None

def validate_text(text):
    if not text:
        print("Error: Input text cannot be empty.")
        return False
    for char in text:
        if char not in alphabet and char != " ":
            print(f"Error: Invalid character '{char}' detected!
               Only Romanian alphabet letters and spaces are
               allowed.")
```

```
            return False
    return True
```

Comment: `validate_key` verifies that the numeric shift is an integer in the range $[1, 30]$, rejecting out-of-bounds keys and non-numeric inputs. `validate_text` iterates through every character, ensuring non-empty input and immediately halting execution if unsupported symbols (e.g. digits or special characters) are detected.

For Task 2, keyword validation verifies both character validity and the minimum length requirement (≥ 7 characters):

```
def validate_keyword(keyword):
    if len(keyword) < 7:
        print("Error: Keyword is too short! It must be at least 7
            characters long.")
        return False
    for char in keyword:
        if char not in alphabet:
            print(f"Error: Invalid character '{char}' in keyword!
                Only Romanian alphabet letters are allowed (no
               spaces or numbers).")
            return False
    return True
```

Comment: Rejects keywords containing fewer than 7 characters or containing non-alphabetic elements, fulfilling the task requirement.

2.3 Building the Permuted Alphabet

Task 2 dynamically constructs the substitution alphabet by extracting unique keyword characters and appending the remainder of the original alphabet:

```
def build_permuted_alphabet(keyword):
    unique_key = []
    for char in keyword:
        if char not in unique_key:
            unique_key.append(char)
    remaining = [char for char in alphabet if char not in
       unique_key]
    return unique_key + remaining
```

Comment: Maintains order of first appearance for keyword characters. The remaining letters of the Romanian alphabet are filtered to avoid duplicates and appended, guaranteeing a bijection of length 31.

2.4 Encryption and Decryption Routines

2.4.1 Task 1: Standard Caesar Cipher

```
def encrypt(text, key):
    result = ""
    for char in text:
        if char in alphabet:
```

```
            new_index = (alphabet.index(char) + key) % len(
                alphabet)
            result += alphabet[new_index]
        else:
            result += char
    print("Encrypted text:", result)
    return result

def decrypt(text, key):
    result = ""
    for char in text:
        if char in alphabet:
            new_index = (alphabet.index(char) - key) % len(
                alphabet)
            result += alphabet[new_index]
        else:
            result += char
    print("Decrypted text:", result)
    return result
```

Comment: Performs direct modular rotation $((i \pm k) \pmod{31})$ over the standard alphabet list.

2.4.2 Task 2: Caesar Cipher with Permutation

```
def encrypt(text, numeric_key, permuted_alphabet):
    result = ""
    for char in text:
        if char in alphabet:
            plain_index = alphabet.index(char)
            cipher_index = (plain_index + numeric_key) % len(
                alphabet)
            result += permuted_alphabet[cipher_index]
        else:
            result += char
    print("Encrypted text:", result)
    return result

def decrypt(text, numeric_key, permuted_alphabet):
    result = ""
    for char in text:
        if char in permuted_alphabet:
            cipher_index = permuted_alphabet.index(char)
            plain_index = (cipher_index - numeric_key) % len(
                alphabet)
            result += alphabet[plain_index]
        else:
            result += char
    print("Decrypted text:", result)
    return result
```

Comment: Encryption looks up the character's position in $\mathscr{A}$, applies the numeric shift k, and extracts the symbol from Π. Decryption locates the ciphertext character in Π, subtracts

$k \pmod{31}$, and recovers the symbol from $\mathscr{A}$.

3 Results

This section presents the actual execution results of both programs, demonstrating encryption, decryption, and error handling for invalid input conditions.

3.1 Task 1: Standard Caesar Cipher Execution

3.1.1 Encryption Demonstration

Figure 1 illustrates the encryption of the plaintext `"securitatea informatiei"` using numeric shift $k = 5$. The program outputs `"vîhzumxdxîd msjșurdxmîm"`.

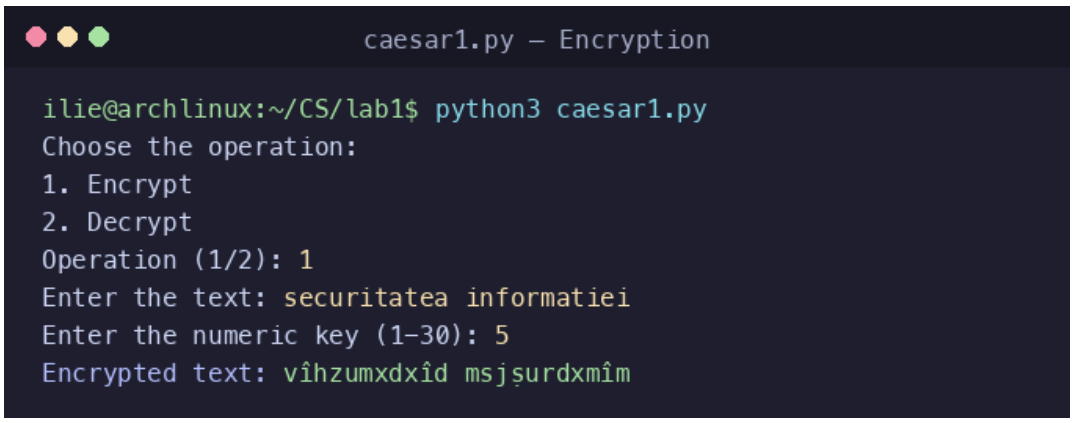


Figure 1: Task 1: Encryption execution using `caesar1.py` ($k = 5$).

3.1.2 Decryption Demonstration

Figure 2 shows the inverse decryption process. The ciphertext `"vîhzumxdxîd msjșurdxmîm"` is decrypted with key $k = 5$, correctly recovering `"securitatea informatiei"`.

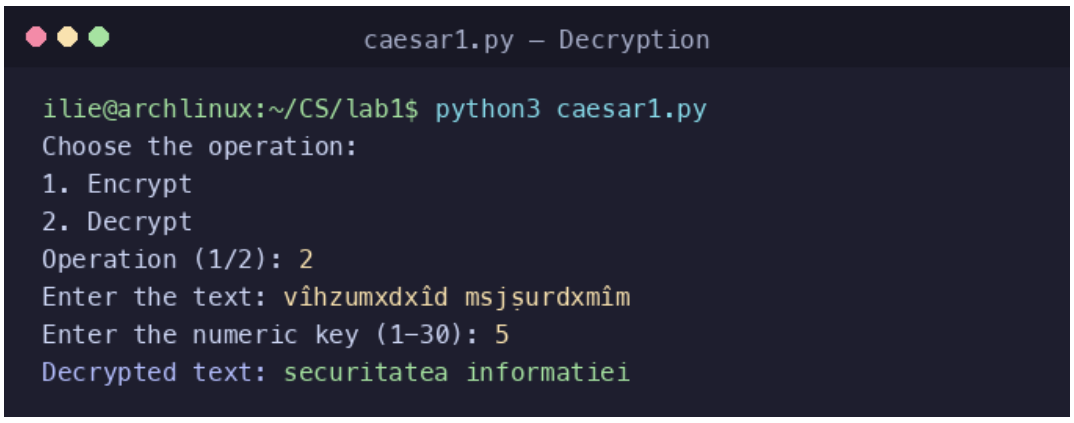


Figure 2: Task 1: Decryption execution using `caesar1.py` ($k = 5$).

3.1.3 Input Validation: Out-of-Bounds Key

When an invalid numeric key outside the range $[1, 30]$ is entered (e.g. $k = 35$), the program detects the error and prints a descriptive error message without terminating abnormally (Figure 3).

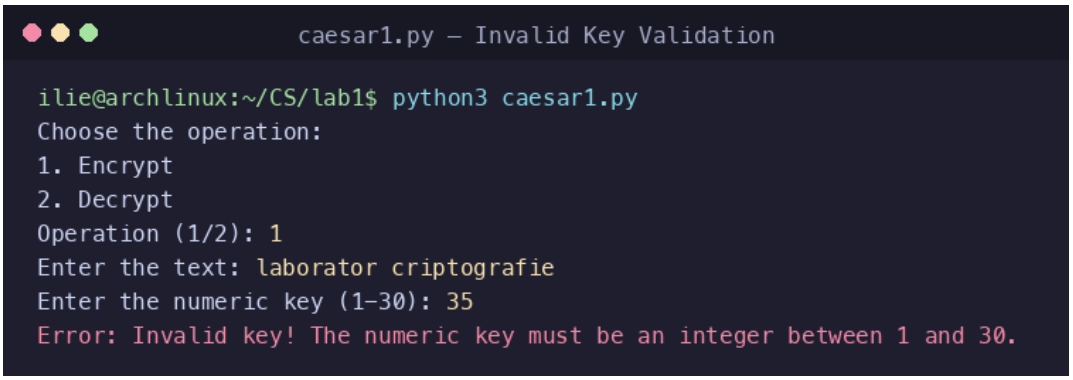


Figure 3: Task 1: Validation message for an out-of-bounds numeric key ($k = 35$).

3.1.4 Input Validation: Invalid Characters

Figure 4 displays the result when the user inputs characters not present in the Romanian alphabet (e.g. digits and exclamation mark in `"parola123!"`).

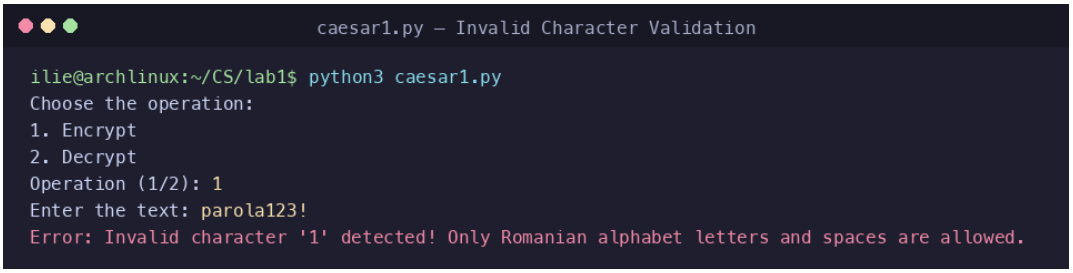


Figure 4: Task 1: Validation error triggered by invalid characters in plaintext.

3.2 Task 2: Caesar Cipher with Permutation Execution

3.2.1 Encryption Demonstration

Figure 5 depicts encryption under Task 2 using numeric shift $k = 7$ and keyword `"universitate"`. The program prints both the standard alphabet and the constructed permuted alphabet, encrypting `"criptografie si securitate"` into `"bwîșzqgwtfîd xî xdbnwîztzd"`.

```
caesar2.py – Permuted Alphabet Encryption

ilie@archlinux:~/CS/lab1$ python3 caesar2.py
Choose the operation:
1. Encrypt
2. Decrypt
Operation (1/2): 1
Enter the text: criptografie si securitate
Enter the numeric key (1-30): 7
Enter the keyword (string key, min 7 chars): universitate
Standard alphabet: aăâbcdefghiiîjklmnopqrsștțuvwxyz
Permuted alphabet: universtaăâbcdfeghîjklmopqșttwxyz
Encrypted text: bwîșzqgwtfîd xî xdbnwîztzd
```

Figure 5: Task 2: Permuted alphabet encryption using `caesar2.py` ($k = 7, K =$ "universitate").

3.2.2 Decryption Demonstration

Figure 6 illustrates the decryption of the ciphertext `"bwîșzqgwtfîd xî xdbnwîztzd"` using the identical key parameters, successfully restoring the original plaintext.

```
caesar2.py – Permuted Alphabet Decryption

ilie@archlinux:~/CS/lab1$ python3 caesar2.py
Choose the operation:
1. Encrypt
2. Decrypt
Operation (1/2): 2
Enter the text: bwîșzqgwtfîd xî xdbnwîztzd
Enter the numeric key (1-30): 7
Enter the keyword (string key, min 7 chars): universitate
Standard alphabet: aăâbcdefghiiîjklmnopqrsștțuvwxyz
Permuted alphabet: universtaăâbcdfeghîjklmopqșttwxyz
Decrypted text: criptografie si securitate
```

Figure 6: Task 2: Permuted alphabet decryption using `caesar2.py`.

3.2.3 Input Validation: Keyword Too Short

When a keyword with fewer than 7 characters is supplied (e.g. `"cheie"`, length = 5), the program rejects the input and prompts the user with an explicit error (Figure 7).

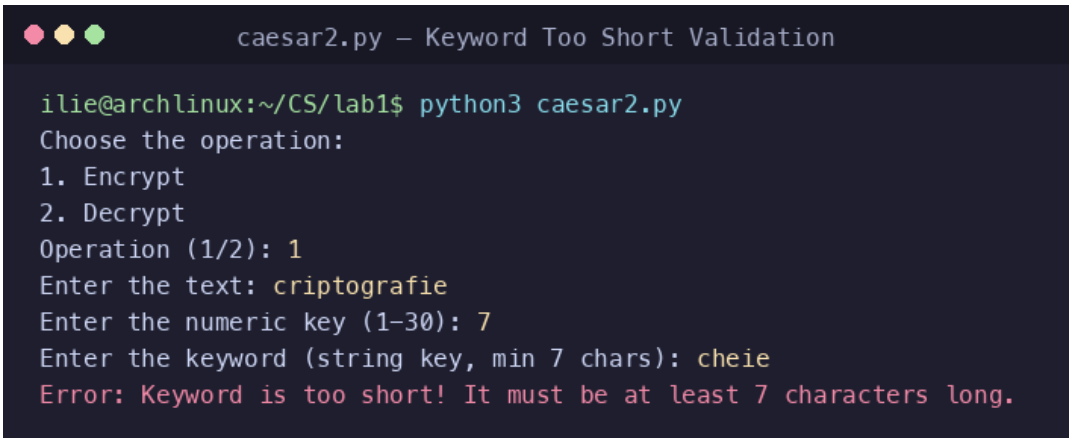


Figure 7: Task 2: Validation message for a keyword that is too short (length < 7).

3.2.4 Input Validation: Invalid Key

Figure 8 shows the rejection of an invalid shift key $k = 0$, which would otherwise produce a degenerate identity mapping.

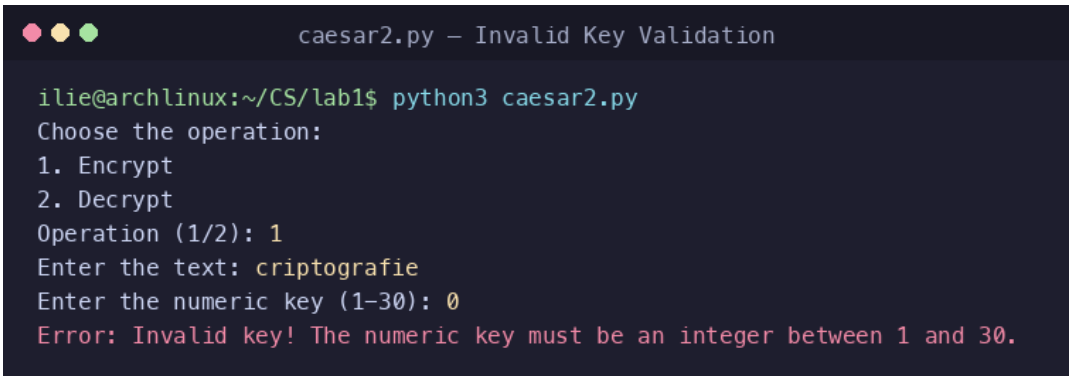


Figure 8: Task 2: Validation message for an invalid shift key ($k = 0$).

4 Conclusions

Through the implementation and analysis of the standard Caesar cipher and the Caesar cipher with a keyword permutation, several key cryptographic conclusions can be drawn:

1. **Vulnerability to Brute Force:** The standard Caesar cipher provides zero practical confidentiality. With an effective key space of only $|\mathscr{K}| = 30$ for the Romanian alphabet, an eavesdropper can try all possible keys in seconds. In contrast, introducing a permuted alphabet generated by a keyword of length ≥ 7 expands the key space to over 3.98×10^{11} combinations, rendering manual exhaustive search impossible.

2. **Persistence of Monoalphabetic Properties:** Despite the massive expansion of the key space in Task 2, both algorithms remain strictly *monoalphabetic substitution ciphers*. Each plaintext symbol maps to exactly one ciphertext symbol for any given key. Consequently, the statistical frequency distribution of the underlying language (such as the high frequency of `e`, `a`, and `i` in Romanian) is preserved in the ciphertext. An attacker with a sufficiently long ciphertext can effortlessly break either cipher using frequency analysis and digram/trigram statistics without ever needing to guess the keyword.

3. **Importance of Rigorous Input Validation:** In practical cryptographic software, defensive programming is vital. Handling diacritics in UTF-8 encoding, enforcing strict bounds on numeric keys, and rejecting malformed inputs prevent runtime crashes and side-channel leakage.

4. **Foundational Learning:** This laboratory work provided valuable hands-on experience with modular arithmetic, bijection mappings, permutation generation, and the fundamental dichotomy between key space size and cryptographic security against statistical attacks.

5 Source Code

The complete source code for this laboratory work, including both implementations, the report generation scripts, and this LaTeX document, is hosted on GitHub:

GitHub Repository Link:
https://github.com/owhowh0/cs-labs/tree/lab1

5.1 Repository File Structure

The repository contains the following structure:

- `caesar1.py`: Implementation of the standard Caesar cipher with input validation.
- `caesar2.py`: Implementation of the Caesar cipher with keyword permutation.
- `generate_screenshots.py`: Automation script generating high-resolution terminal execution captures.
- `report/`:
  - `report.tex`: The complete LaTeX source code for this report.

  - `labreport.cls`: Custom university report document class.
  - `images/`: Screenshot assets documenting program execution.

- `pyproject.toml`: Project metadata and configuration.